

Modifikovaný ERC20 token

BDA projekt 2024/25

Konzultant: Samuel Oleksák (ioleksak@fit.vut.cz)

Posledná aktualizácia: 20. februára 2025

1 Súhrn zadania

Tento projekt slúži na precvičenie činností typicky asociovaných s vytváraním blockchainovej aplikácie na EVM (Ethereum Virtual Machine) kompatibilnej platforme (ako Ethereum, Polygon, Hyperledger Besu, atď.). Projekt zahŕňa vytvorenie smart contractu v jazyku Solidity, ktorý implementuje ERC20 token s rozšírenou funkcionalitou. Zadanie zahŕňa aj implementáciu dApp, ktorá umožní pohodlnú interakciu s contractom pomocou grafického rozhrania.

Znalosti potrebné na vypracovanie zadania získate na prednáškach o platforme Ethereum (5. týždeň semestra), o programovaní smart contractov (6. týždeň semestra) a demonštračnom cvičení (7. týždeň semestra).

2 Štruktúra zadania a bodovanie

Hodnotenie riešenia sa delí do štyroch kategórií – hodnotenie smart contractu, hodnotenie dApp, hodnotenie testovania a hodnotenie dokumentácie. Zadanie je rozdelené do úloh, pričom každá ďalšia úloha do výsledného produktu pridáva funkcionalitu. Každá úloha má určený počet bodov, ktorý sa dá za jej splnenie získať. Tento počet bodov je proporcionálny náročnosti úlohy.

Úlohy sú navrhnuté tak, aby ste si mohli vybrať oblasti, ktoré vás viac zaujímajú, a preto sa nutne neočakáva, že vypracujete každú úlohu – maximálny počet bodov z projektu (40) môžete získať aj vypracovaním podmnožiny všetkých úloh. Za výborné riešenie, ktoré presahuje hranicu 40 bodov, môžu byť udelené bonusové body.

Na získanie minimálneho počtu bodov z projektu je nutné splniť úlohu 1.1 označenú ako povinnú. Niektoré úlohy na seba nadväzujú a v tom prípade sú návazné úlohy označené oranžovým textom s číslom úlohy, ktorá musí byť predom vypracovaná.

3 Úlohy

3.1 Smart contract

Úloha 1.1 Implementujte smart contract, ktorý rozširuje ERC20 token a **povinná** obmedzuje maximálny obnos tokenov v cirkulácii na hodnotu *2 body* `maxSupply` zadanú v konštruktore.

Úloha 1.2 Implementujte rolu `mintingAdmin`. Počiatoční členovia role budú definovaní argumentom konštruktora (pole adres). Členovia role budú môcť mintovať nové tokeny na nimi zvolenú adresu, avšak súčet vymintovaných tokenov za deň nesmie presiahnuť hodnotu parametra `maxDailyMint` definovanom v konštruktore.

Úloha 1.3 Implementujte rolu `restrAdmin`. Počiatoční členovia role budú definovaní argumentom konštruktora (pole adres). Členovia role budú môcť nastaviť pre vybrané adresy denný limit na transfer tokenov. Každá adresa môže mať určený iný limit.

Úloha 1.4 Implementujte verifikáciu identity pre držiteľov tokenov. Tokeny budú môcť držať iba adresy, ktoré boli verifikované jedným z centralizovaných poskytovateľov identity (identity provider – IDP), ktorí sú identifikovaní svojou adresou. Počiatočný zoznam IDP bude definovaný argumentom konštruktora (pole adres). Do svojho riešenia pridajte skript (mimo smart contractu), ktorý bude simulovať centralizovaného poskytovateľa identity, viď sekcia 4.4.

Úloha 1.5 Implementujte rolu `idpAdmin`. Počiatoční členovia role budú definovaní argumentom konštruktora (pole adres). Členovia role budú **po 1.4** môcť pridávať alebo odoberať položky zo zoznamu centralizovaných poskytovateľov identity. *2 body*

Úloha 1.6 Implementujte expiráciu validity identity jednotlivých adres s overenou identitou. Hodnota `expirationTime` bude definovaná v konštruktore a bude udávať počet hodín od získania potvrdenia od centralizovaného poskytovateľa identity, po koľkých verifikácia vyprší (a identita vlastníka adresy bude musieť byť znovu overená). Adresy, ktoré stratili verifikáciu nebudú do reverifikácie môcť manipulovať so svojimi tokenami. **po 1.4** *2 body*

- Úloha 1.7** Rozšírte rolu `idpAdmin`. Každý člen bude môcť manuálne verifikovať ľubovoľnú adresu bez nutnosti použiť centralizovaného poskytovateľa identity alebo bude môcť revokovať verifikáciu adresy. Revokovaná adresa stratí ihneď validitu, ale môže sa pomocou IDP reverifikovať. Každý člen role taktiež bude môcť zablokovať alebo odblokovať ľubovoľnú adresu. Zablokovaná adresa nestratí ihneď validitu, ale do odblokovania nemôže manipulovať so svojimi tokenami.
- po 1.5**
2 body
- Úloha 1.8** Smart contracty budú emitovať eventy pri dôležitých udalostiach, napr. `TokensMinted`, `TokensTransferred`, `TransferRestrictionCreated`.
- 1 bod
- Úloha 1.9** Umožnite všetkým implementovaným rolám, aby nadpolovičnou väčšinou hlasov mohli pridávať alebo odoberať členov svojej role (tzn. prahová schéma).
- 3 body

3.2 dApp

- Úloha 2.1** Implementujte dApp, ktorá umožní používateľovi pripojenie peňaženkovej aplikácie *MetaMask* a poskytne grafické používateľské rozhranie ukazujúce adresu aktuálne zvolenej pripojenej adresy, aktuálny obnos tokenov a umožní transfer zadaného množstva tokenov na zadanú adresu.
- 4 body
- Úloha 2.2** dApp poskytne grafické používateľské rozhranie umožňujúce schválenie minútia zadanej čiastky zadanou adresou (approve funkcionality ERC20).
- 1 bod
- Úloha 2.3** dApp poskytne grafické používateľské rozhranie umožňujúce minovanie nových tokenov pre členov role `mintingAdmin` a ukazujúce koľko tokenov sa smie ešte daný deň vymintovať.
- po 1.2**
1 bod
- Úloha 2.4** dApp poskytne grafické používateľské rozhranie umožňujúce vytvorenie a následné zrušenie obmedzení na transfer pre členov role `transferAdmin`.
- po 1.3**
1 bod
- Úloha 2.5** dApp poskytne grafické používateľské rozhranie umožňujúce verifikáciu identity vlastníka aktuálne zvolenej adresy. V prípade, že aktuálne zvolená adresa nie je verifikovaná, tak rozhranie dApp neumožní manipulovať s tokenami (schovanie alebo deaktivácia prvkov UI).
- po 1.4**
2 body

- Úloha 2.6** dApp poskytne grafické používateľské rozhranie umožňujúce pridanie alebo odobratie adres dôveryhodných IDP členom role `idpAdmin`.
po 1.5
 1 bod
- Úloha 2.7** dApp poskytne grafické používateľské rozhranie zobrazujúce ostávajúci čas do expirácie validity identity a umožní (aj predbežnú) reverifikáciu identity.
po 1.6
 1 bod
- Úloha 2.8** dApp poskytne grafické používateľské rozhranie umožňujúce členom role `idpAdmin` manuálne verifikovať adresy, revokovať expiráciu adres, blokovat adresy a odblokovat adresy.
po 1.7
 2 body
- Úloha 2.9** dApp bude načúvať na eventy emitované smart contractom a pri prijatí eventu obnoví zobrazené údaje v dApp; a teda nie je potrebný manuálny zásah používateľa.
po 1.8
 2 body
- Úloha 2.10** dApp umožní všetkým implementovaným rolám vytvárať hlasovania o pridávaní alebo odoberaní členov role a pridávať hlasy v existujúcich hlasovaniach.
po 1.9
 2 body

3.3 Dokumentácia

- Úloha 3.1** Stručne popíšte v dokumentácii zvolené implementačné technológie a použité knižnice. Popíšte kroky potrebné k spusteniu lokálneho prostredia (spustenie lokálneho blockchainu, nasadenie smart contractu, spustenie dApp). Popíšte hlavné časti vášho riešenia a zdôvodnite rozhodnutia, ktoré ste pri implementácii učinili. Ak ste riešenie rozdelili na viacero smart contractov, tak jednotlivé contracty popíšte. Popíšte vašu dApp.
 1 až 4 body
- Úloha 3.2** Vytvorte súbor `Makefile` na jednoduchšiu manipuláciu s projektom s nasledujúcimi cieľmi:
1. `network` – spustí lokálny blockchain.
 2. `deploy` – nasadí smart contract na lokálny blockchain.
 3. `dapp` – spustí dApp.
 4. `test` – spustí testy smart contactu.¹
- Úloha 3.3** Popíšte ako ste implementovali resetovanie denných limitov o polnoci, popíšte výhody a nevýhody vami zvoleného prístupu oproti iným.
po 1.2
 1 bod

Úloha 3.4 Analyzujte množstvo gasu spotrebovaného na vykonanie základných operácií contractu (transfer tokenov, vytvorenie návrhu hlasovania, zahlasovanie za návrh, ...). Zamerajte sa na najdrahšiu operáciu, ktorú nájdete a vysvetlite, prečo využíva toľko gasu.
2 body

Úloha 3.5 Popíšte svoje stratégie a prekonané problémy pri testovaní. Popíšte techniky (napr. test driven development), ktoré ste používali; popíšte ako sú vaše testy členené na menšie celky; popíšte ako ste zasahovali do EVM (simulovanie prebehnutého času, atp.); popíšte ako ste zachytávali chyby a aké typy chýb ste v priebehu písania testov objavili.
1 bod

Úloha 3.6 Modelujte konečným automatom stavy, ktoré môže dosiahnuť adresa v priebehu svojho života vzhľadom k verifikáciám, revokáciám a zablokovaniu. Stavy nazvite, aby zodpovedali premenným v kóde smart contractu a k hranám uveďte metódy smart contractu, ktoré môžu tento prechod spôsobiť.
po 1.7
1 bod

3.4 Testovanie

Úloha 4.1 Verifikujte pomocou testov implementáciu úloh z kategórie smart contract. Pri písaní testov sa nezamerajte iba na testovanie úspešného vykonávania (happy paths), ale aj na testovanie správneho ošetrovania chýb (exception handling) a testovanie krajných prípadov (edge cases).
1 až 4 body

Úloha 4.2 Nájdite vhodné miesto, kde by sa v testoch smart contractu dal použiť fuzzing a doplňte ho. V dokumentácii popíšte, prečo je práve tam vhodný fuzzing.
2 body

Úloha 4.3 Skontrolujte si kód vášho smart contractu pomocou nástroja na statickú analýzu Solidity (napr. *slither*), uveďte do dokumentácie chyby, aké ste ním odhalili.
1 bod

Úloha 4.4 Vytvorte E2E (end-to-end) testy, ktoré lokálne spustia blockchain, nasadia naň smart contracty a automatizovane otestujú základnú funkcionálnosť používateľského rozhrania dApp (napr. vloženie adresy do textového poľa a stlačenie tlačidla „Mint“ spôsobí aktualizáciu hodnoty na konte danej adresy).
4 body

4 Implementačné detaily

V projekte je povolené (a aj odporúčané) využiť kostru projektu z demonstračného cvičenia, ktoré bude predvedené v 7. týždni semestra. V prípade záujmu o použitie iných ako vymenovaných knižníc sa predom poraďte s konzultantom projektu. Nepoužívajte nástroje a knižnice, ktoré sú ich vývojarmi označené za zastaralé (deprecated/sunset), ako sú napr. *web3.js*, *Ganache*, *Truffle*.

4.1 Smart contracty

Ako prostredie pre vývoj, nasadenie a testovanie smart contractov si môžete vybrať *Foundry* (odporúčané) alebo *Hardhat*. Obe prostredia umožňujú kompiláciu a testovanie smart contractov a spustenie lokálneho blockchainu, ktorý poskytuje ladiace nástroje na zjednodušenie vývoja.

	<i>Hardhat</i>	<i>Foundry</i>
Jazyk, v ktorom bola knižnica vyvinutá	JavaScript	Rust
Vývoj smart contractu	Solidity	Solidity
Skripty na nasadenie contractu	JavaScript	Solidity
Testovanie contractu	JavaScript	Solidity

Tabuľka 1: Porovnanie jazykov používaných na vývoj v prostrediach.

Smart contracty vyvíjajte v Solidity vo verzii 0.8.28 alebo vyššej. Pri vypracúvaní väčšieho množstva úloh z kategórie „Smart contract“ bude pravdepodobne vhodné rozdeliť funkcionality do viacerých smart contractov, ktoré spolu komunikujú. Pri vývoji je odporúčané využívať existujúce komponenty z knižnice *OpenZeppelin*². Vhodné použitie viacerých komponentov môže byť hodnotené bonusovými bodmi.

4.2 dApp

Používateľské rozhranie (UI) dApp môže byť základné so surovými HTML prvkami bez štýlov, avšak prehľadné a intuitívne rozhranie so správnym ošetrovaním chýb môže byť hodnotené bonusovými bodmi. V dokumentácii môžete spomenúť, aké kroky pri vývoji UI ste podnikli, aby ste prehľadnosť a intuitívnosť dosiahli.

dApp vyvíjajte v prostredí *Node.js* a na interakciu s blockchainom použite knižnicu *Wagmi* alebo *ethers.js* a ako peňaženkový softvér *MetaMask*. Použitie UI frameworku alebo knižnice ako napr. *React.js* je povolené. Na vytvorenie dApp je zakázané používať knižnice, ktoré dokážu automaticky vygenerovať dApp z kódu smart contractu, ako napr. *Scaffold-ETH*.

²<https://github.com/OpenZeppelin/openzeppelin-contracts>

4.3 Dokumentácia

Dokumentácia je obsiahnutá v odovzdanom dokumente vo formáte PDF (.pdf) alebo Markdown (.md) a môže byť vypracovaná v anglickom, českom alebo slovenskom jazyku. Do dokumentácie uveďte číselné označenie úloh, ktoré ste zo zadania vypracovali.

4.4 Verifikácia identity

Niektoré úlohy zahŕňajú využitie centralizovaného poskytovateľa identity (IDP). Centralizované IDP fungujú mimo blockchainu a bývajú implementované pomocou asymetrickej kryptografie. Pri úspešnom overení identity IDP podpíše správu v predpísanom tvare, napr. „User with address <address> has verified their identity at <UNIX timestamp>“.

Pri verifikácii používateľ smart contractu poskytne časovú značku verifikácie (adresa je prístupná z `msg.sender`). Smart contract zostaví správu v predpísanom tvare a použije vstavanú funkciu `ecrecover`, aby zistil, či verejný kľúč podpisovateľa správy zodpovedá jednému z IDP, ktorým dôveruje.

5 Odovzdávanie

Projekt odovzdávajte do STUDISu vo formáte ZIP pomenovanom vaším loginom. Do ZIP archívu nedávajte súbory, ktoré sa sťahujú alebo generujú automaticky – ABI smart contractov, binárne súbory, npm balíčky (pričínok `node_modules/`), atp. Deadline na odovzdanie projektu je 1. mája 2025 23:59.

6 Doplnujúce poznámky

Pri vypracovaní projektu je zakázané používať nástroje generatívnej umelej inteligencie (ChatGPT, DeepSeek, Github Copilot, ...) na generovanie kódu alebo dokumentácie.

V prípade, že máte nápady na ďalšie úlohy, ktoré by rozumnne vedeli rozšíriť projekt, tak ich môžete prebrať s konzultantom projektu. Zaujímavé úlohy môžu dodatočne byť pridané do zadania a ich autor môže byť ocenený bonusovými bodmi.